

CSCI 650 - Algorithms and Computability

Assignment 4

Solutions to the written questions on this assignment should be submitted via PDF to Canvas before October 13th at 11:59 pm. Implementations for question 1 should be submitted to INGINious by the same deadline. Make sure to justify your answers.

I highly encourage you to collaborate with one another. However, you must write up your own solutions independently. Feel free to communicate via Discord and to post questions on the appropriate forum in Canvas. Do not post solutions.

Have fun!

Problems

1. In this problem we will develop and analyze two heuristic approaches to solving the Traveling Salesman problem, one using nearest neighbors and the other using simulated annealing.

a. (5 pts) Write a function `totalDistance(D, path)` which returns the total distance a traveling salesman must travel given a sequence of cities. `path` will include each city exactly once, note that the salesman must end at the start city. Submit your code on INGINious and include a code snippet of this function in your PDF submission.

Answer.

The route lists each city once. Sum the directed distances between consecutive cities and include the final edge back to the first city. Empty routes have distance zero. The calculation takes $O(n)$ time and $O(1)$ auxiliary space.

```
def totalDistance(D, path):
    if not path:
        return 0.0
    return float(sum(D[path[j]][path[(j + 1) % len(path)]]
                     for j in range(len(path))))
```

b. (10 pts) Write a function `nearestNeighbor(D, start)` which returns an order in which to visit cities with associated distance matrix `D` starting at `start` and following the nearest neighbor heuristic. Submit your code on INGINious and include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function?

Answer.

Start at the requested city, or choose a random city when `start` is `-1`. Repeatedly select the closest unvisited city; ties go to the smaller city index. The returned route contains each city exactly once.

```
def nearestNeighbor(D, start=-1):
    n = len(D)
    if n == 0:
```

```

    return []
if start == -1:
    start = int(np.random.randint(n))
path = [start]
remaining = set(range(n))
remaining.remove(start)
while remaining:
    city = min(remaining, key=lambda v: (D[path[-1]][v], v))
    path.append(city)
    remaining.remove(city)
return path

```

At successive steps the algorithm examines $n-1$, $n-2$, ..., 1 remaining cities. Their sum is $n(n-1)/2$, so the runtime is $\Theta(n^2)$, assuming constant-time matrix access and expected constant-time set operations. Auxiliary space is $O(n)$. This greedy rule does not guarantee an optimal tour.

c. (5 pts) Use the `tspTests.py` script provided on Canvas to generate histograms depicting the distribution of ratios (observed distance)/(minimum observed distance) under different circumstances. Describe what you observe.

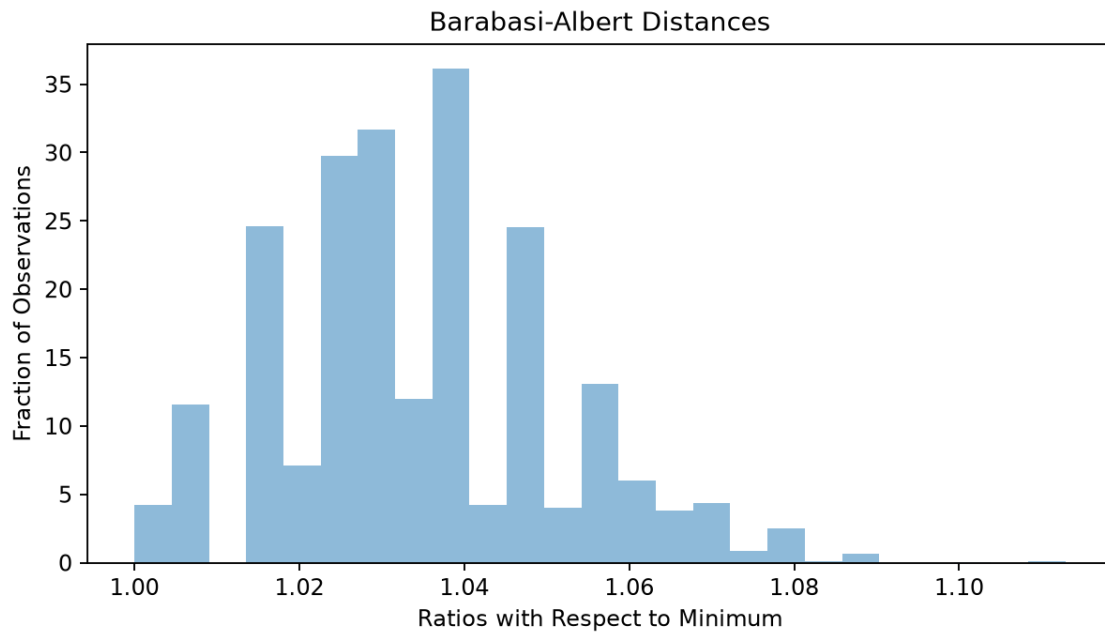
Answer.

The supplied tests used NumPy and Python random seeds of 65004. Each of three families used 50 independently generated distance matrices with 100 cities. Each matrix was tested at every starting city, yielding 5,000 ratios per family and 25 histogram bins. Each denominator is the minimum nearest-neighbor tour observed on that matrix, not a proven optimal TSP distance.

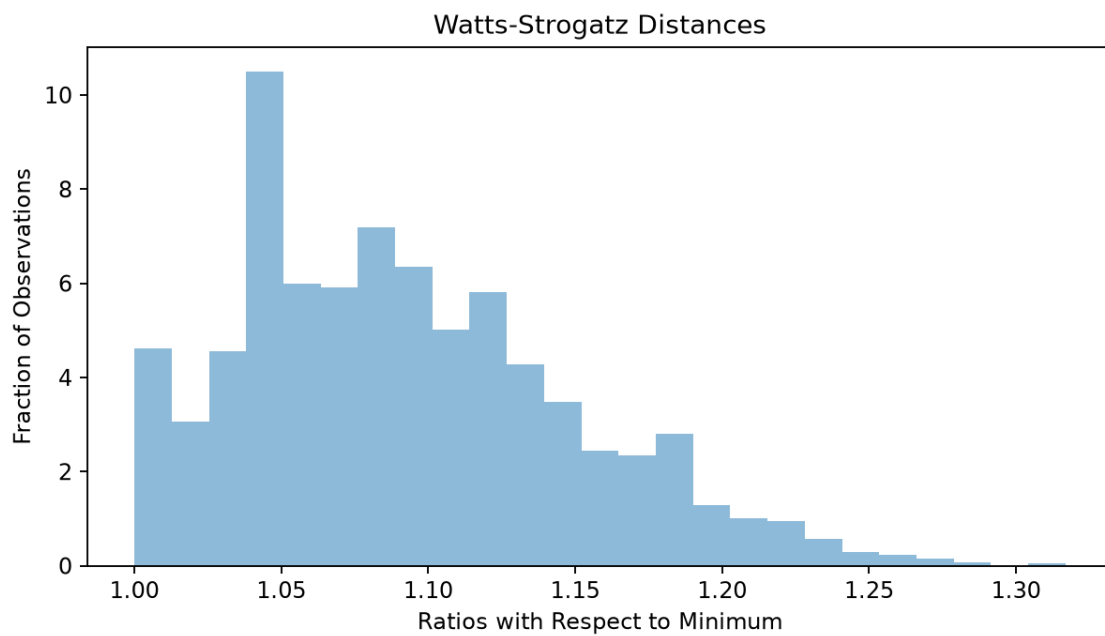
Family	Mean	Median	95th percentile	Maximum
Barabasi-Albert	1.034	1.032	1.065	1.113
Watts-Strogatz	1.092	1.085	1.198	1.317
Uniform Random	1.421	1.411	1.772	2.188

Barabasi-Albert ratios cluster tightly near 1: the average tour is about 3.45% longer than the best start for its matrix. Watts-Strogatz has a wider distribution, averaging about 9.20% above its best start. Uniform Random is much more sensitive to start: the mean is about 42.09% above the best observed start, with a maximum ratio of 2.188. The uniform matrices are asymmetric and need not satisfy the triangle inequality; an inexpensive greedy edge can leave an expensive closing edge.

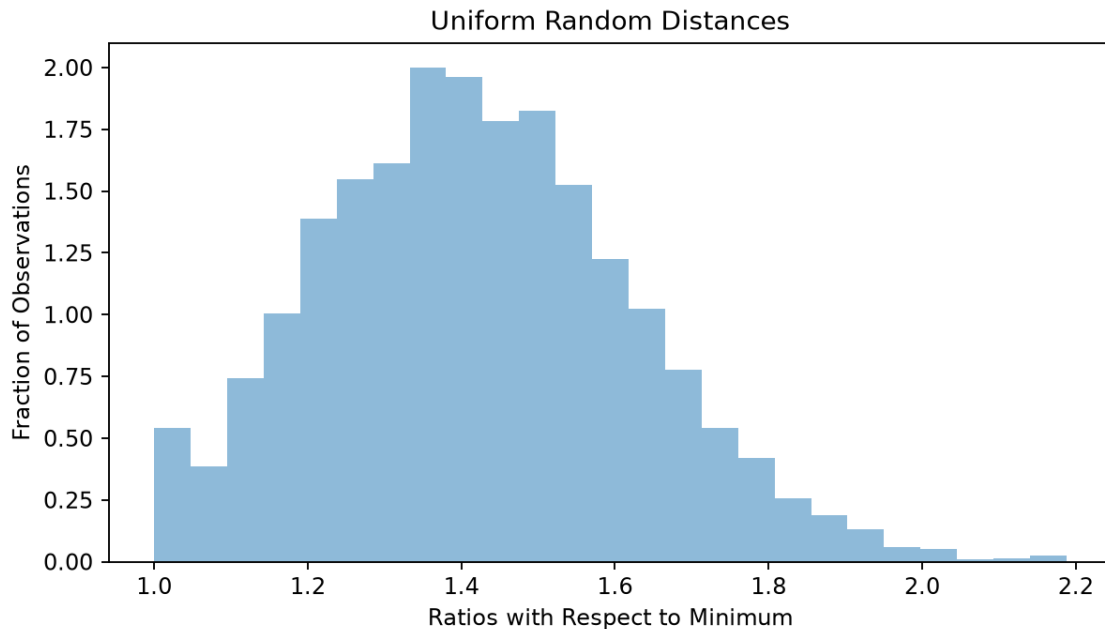
All starts on one matrix share its denominator and are dependent observations. These histograms describe these sampled instances; they do not establish approximation guarantees. Trying several starts can improve the observed nearest-neighbor result.



Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.



Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.



Horizontal axis: tour distance / minimum distance on the same matrix. Vertical axis: probability density, matching density=True in the supplied script.

d. (5 pts) Write three functions to generate candidate solutions. - swapPairs(path, numSwaps, adj) swaps numSwaps random pairs of elements in path. If adj is True, only swap adjacent pairs of elements in the current path. - permuteRange(path) randomly permutes a slice of path between two randomly selected indices. - reverseRange(path) reverses a slice of path between two randomly selected indices. Include code snippets of these functions in your PDF submission.

Answer.

Each function copies its input and preserves the route's city multiset. For adjacent swaps I use consecutive list indices, without wrapping the last index to the first. The range functions choose distinct endpoints and include both endpoints. Paths with fewer than two cities return a copy unchanged.

```
def swapPairs(path, numSwaps=1, adj=False):
    candidate = path[:] #we do not want to change the given path
    n = len(candidate)
    if n < 2:
        return candidate
    for _ in range(numSwaps):
        if adj:
            a = int(np.random.randint(n - 1))
            b = a + 1
        else:
            a, b = np.random.choice(n, size=2, replace=False)
```

```

        candidate[a], candidate[b] = candidate[b], candidate[a]
    return candidate

def permuteRange(path):
    candidate = path[:] #we do not want to change the given path
    if len(candidate) < 2:
        return candidate
    a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
    candidate[a:b + 1] = np.random.permutation(candidate[a:b + 1]).tolist()
    return candidate

def reverseRange(path):
    candidate = path[:] #we do not want to change the given path
    if len(candidate) < 2:
        return candidate
    a, b = sorted(np.random.choice(len(candidate), size=2, replace=False))
    candidate[a:b + 1] = candidate[a:b + 1][::-1]
    return candidate

```

A random permutation may return the original slice; multiple swaps may undo one another. Such proposals remain valid. With a fixed number of swaps, all three functions take $O(n)$ time because they copy the route. NumPy choice without replacement may construct a permutation of the population, so a conservative bound for swapPairs is $O(n \cdot (1 + \text{numSwaps}))$ in general. Each uses $O(n)$ auxiliary space.

e. (10 pts) Write a function runSimulatedAnnealing(D, genCandidate, maxIterations, T, alpha) which finds a path for a traveling salesman using simulated annealing given a distance matrix D, a function to generate candidate solutions, a maximum number of iterations to perform, an initial temperature, and a multiplicative update factor for temperature. The initial path should be a random permutation of the cities. Include a code snippet of this function in your PDF submission. What is the asymptotic runtime of this function with respect to n (the number of cities) and i (the number of iterations)?

Answer.

Initialize with a random permutation. Always accept a candidate of equal or lower distance. Accept an uphill change of size delta with probability $\exp(-\text{delta}/T)$ when T is positive. Multiply T by alpha after each iteration. Keep the best-ever tour separately from the current tour; uphill moves may worsen the current tour. At $T=0$ only non-increasing moves are accepted. distList records the current distance after each iteration, and has exactly maxIterations entries.

```

def runSimulatedAnnealing(D, genCandidate=swapPairs, maxIterations=1000, T=-1, alpha=0.9):
    if T==-1: T = np.sqrt(len(D)) #default starting temperature
    T = float(T)
    curPath = np.random.permutation(len(D)).tolist() #a random initial path
    curDist = totalDistance(D, curPath)
    bestPath, bestDist = curPath[:], curDist
    distList = []
    for _ in range(maxIterations):
        candidate = genCandidate(curPath[:])
        candidateDist = totalDistance(D, candidate)
        delta = candidateDist - curDist
        if delta <= 0 or (T > 0 and np.random.random() < np.exp(-delta / T)):
            curPath, curDist = list(candidate), candidateDist
            if curDist < bestDist:
                bestPath, bestDist = curPath[:], curDist
            distList.append(curDist)
        T *= alpha

```

```
return bestPath, bestDist, distList
```

For the provided candidate functions, candidate construction, tour scoring, and any route copy each take $O(n)$ time. Thus i iterations take $O(n + i*n)$, or $O((i+1)*n)$, for n at least 1. Including degenerate empty inputs gives $O((i+1)*(n+1))$. Auxiliary space is $O(n+i)$, including the returned distance history. With a general generator taking $G(n)$ time, the bound becomes $O(n + i*(n+G(n)))$.

f. (5 pts) Use the `tspTests.py` script provided on Canvas to generate plots showing how the total distance of the current solution changes over time. Discuss the results and briefly describe a process for tuning the hyperparameters of this approach.

Answer.

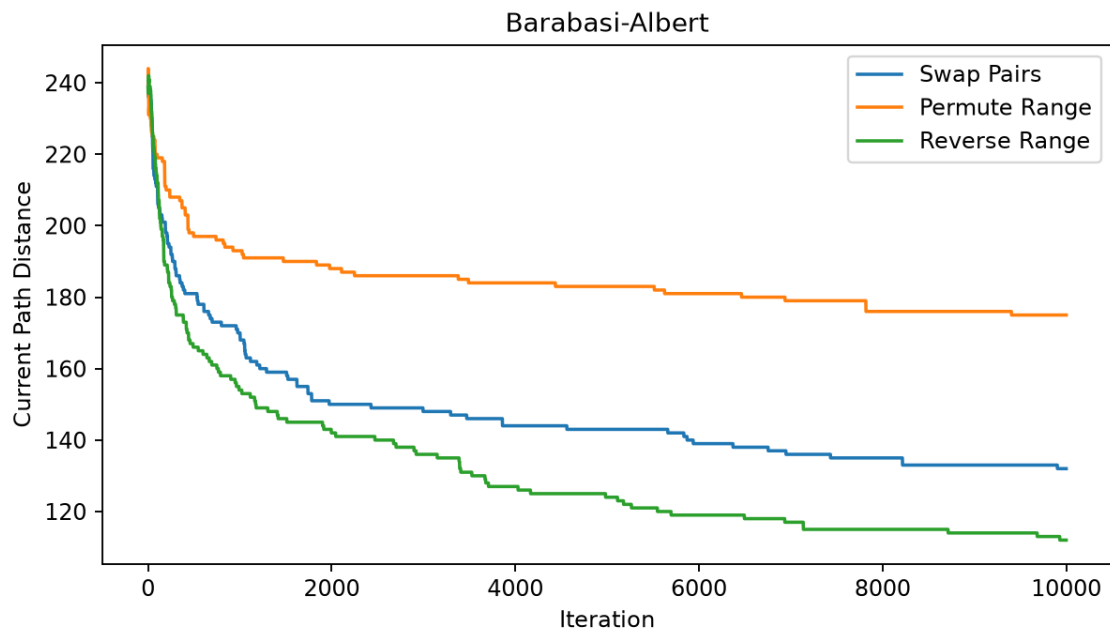
For each family the supplied script generates one 100-city matrix and runs each candidate function for 10,000 iterations on that same matrix. Each run has its own random initial tour, $T=10$, and $\alpha=0.9$. These are single runs per combination, so the comparison is descriptive and is confounded by different initial tours.

Family	Generator	Initial	Best / final
Barabasi-Albert	swapPairs	239.000	132.000
Barabasi-Albert	permuteRange	233.000	175.000
Barabasi-Albert	reverseRange	236.000	112.000
Watts-Strogatz	swapPairs	494.000	176.000
Watts-Strogatz	permuteRange	499.000	274.000
Watts-Strogatz	reverseRange	529.000	121.000
Uniform Random	swapPairs	52.781	10.147
Uniform Random	permuteRange	51.351	19.597
Uniform Random	reverseRange	49.458	23.863

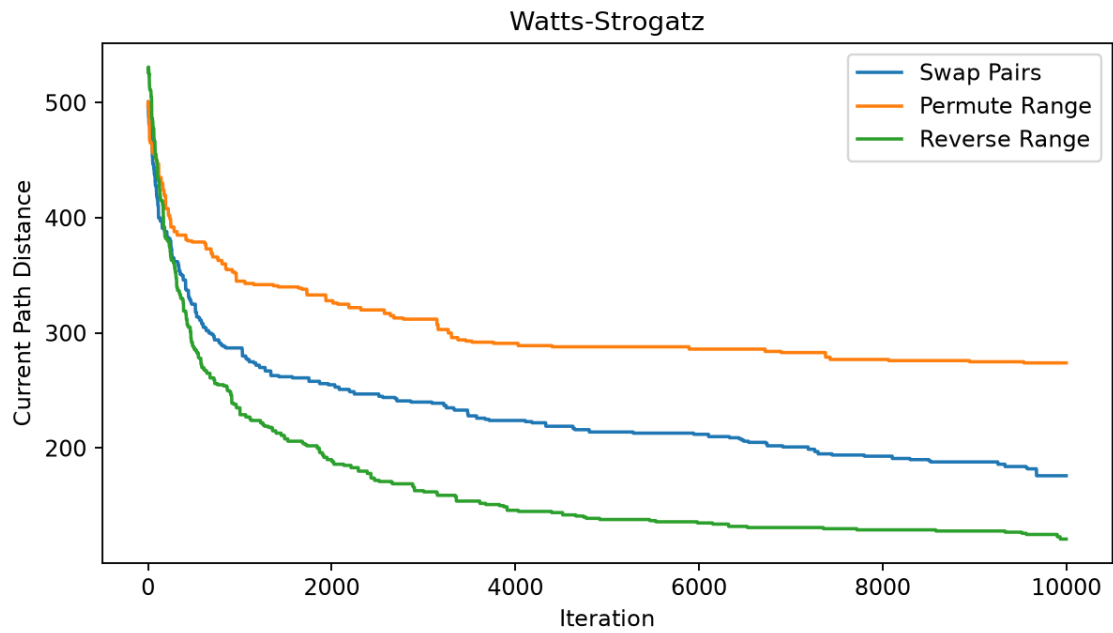
All runs improve substantially and end at their best recorded distances in this experiment. Reversal performs best on the two symmetric graph-distance instances (112 and 121), while pair swaps perform best on the asymmetric uniform instance (10.147). In a symmetric matrix, reversing a segment preserves its internal edge costs and changes just the boundary edges, resembling a 2-opt move. In an asymmetric matrix, reversal also reverses all its internal arcs and can be much more disruptive. Large random permutations often disturb many good edges.

The plotted current distance can rise during early uphill acceptances, but settles into a mostly decreasing staircase as cooling proceeds. The default cools rapidly: T after 100 updates is about 0.000266, and after 200 about 0.00000000706. Almost all the 10,000 iterations therefore behave like greedy local improvement. A late plateau indicates that accepted improvements are rare under this schedule; it does not certify optimality.

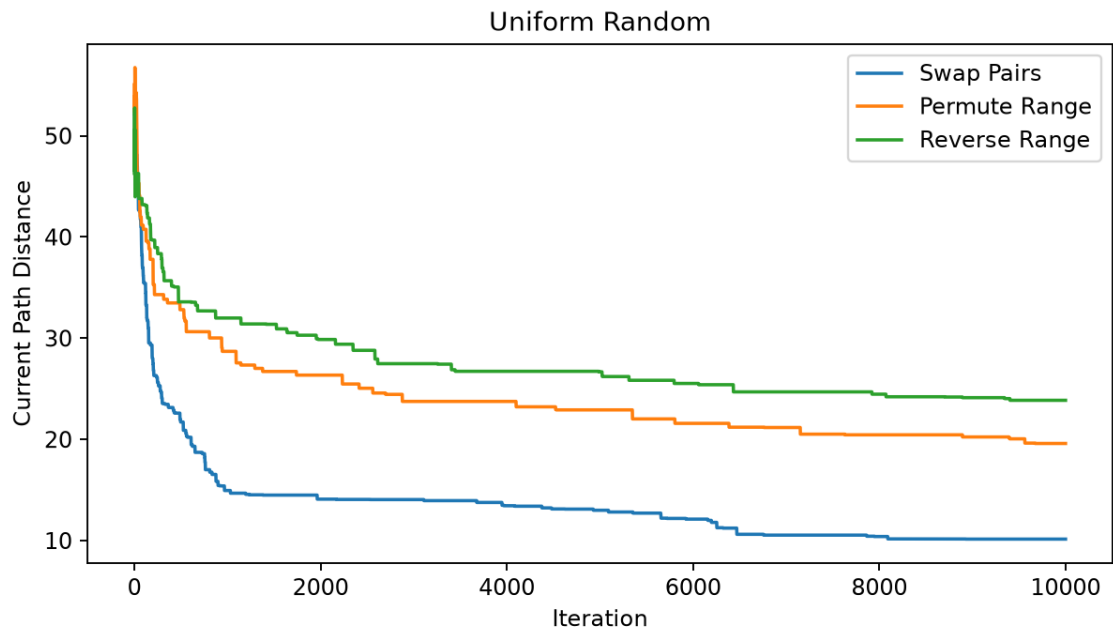
To tune, first sample positive candidate distance changes and choose T_0 near $-\text{delta_typical} / \log(p_0)$, for example $p_0=0.8$. Choose a final temperature and set $\alpha=(T_{\text{final}}/T_0)^{1/i}$ so cooling spans the actual iteration budget. Compare multiple candidate operators, temperatures, schedules, and restart counts using equal evaluation budgets and repeated paired seeds on the same matrices and initial tours. Compare the distribution of best distances and runtime, then evaluate the chosen setting on fresh instances. Slower cooling may help escape local minima but is not guaranteed to improve every run.



Horizontal axis: iteration. Vertical axis: current closed-tour distance.



Horizontal axis: iteration. Vertical axis: current closed-tour distance.



Horizontal axis: iteration. Vertical axis: current closed-tour distance.

2. Let us return to the busy beaver numbers.

a. (5 pts) Describe why the busy beaver numbers are incomputable intuitively. What are the practical consequences of this with respect to determining $BB(n)$?

Answer.

Use the course's step-count convention: $BB(n)$ is the largest number of steps taken before halting by any n -working-state, binary-tape Turing machine started on an all-zero tape; a separate halt state is not counted. This is consistent with the previous assignment's $BB(2)=6$ convention. It is not the alternative definition that maximizes the number of printed ones.

For each fixed n there are finitely many machines, and finitely many halting times among those that halt, so this maximum is finite. However, simulating all those machines longer and longer cannot in general distinguish a machine that runs forever from one that eventually halts after an enormous delay. A computable maximum halting-time bound would remove that difficulty and decide halting, which is impossible.

Consequently there is no algorithm guaranteed to compute $BB(n)$ for every n . Halting examples establish lower bounds, and longer searches may improve them. A finite cutoff cannot generally prove that all remaining machines fail to halt. An exact value requires ruling out every competing machine above the claimed maximum, typically through proofs of nontermination or bounded halting behavior. Some individual small values can be established; incomputability concerns one uniform algorithm for all n .

b. (10 pts) Show that the busy beaver numbers are incomputable via reduction from a problem discussed in class. In particular, write pseudocode (using Python syntax) showing that a known undecidable problem can be solved assuming the existence of a function $BB(n)$ which is guaranteed to halt.

Answer.

Assume for contradiction that $BB(n)$ is computable and always halts. Given an arbitrary machine M and input w , effectively compile a binary-tape, blank-start machine N that first writes an encoding of w , then simulates M on w . The compiler uses finitely many additional working states and preserves halting: N halts on its blank tape if and only if M halts on w . Count the working states of this constructed machine; do not use the unmodified state count of M .

```
def decideHalting(M, w):
    N = compileToBinaryBlankStart(M, w)
    n = numberOfWorkingStates(N)
    limit = BB(n)
    config = blankConfiguration(N)
    if isHalted(config):
        return True
    for _ in range(limit):
        config = step(N, config)
        if isHalted(config):
            return True
    return False
```

The initial halt check handles zero steps, and checking after each transition includes halting at exactly limit steps. The hypothetical BB call terminates, and the loop is bounded, so `decideHalting` always terminates. If

N halts, its halting time cannot exceed $BB(n)$, and the simulation accepts. If it has not halted within that bound, it cannot halt later without contradicting the definition of $BB(n)$, and the function rejects. Therefore this procedure would decide the general halting problem. Since that problem is undecidable, a total computable BB function cannot exist.

Even restricting to blank-input machines is already undecidable: the input-writing wrapper reduces arbitrary-input halting to blank-input halting. If a different source uses the printed-ones Busy Beaver definition, this particular time-bound loop needs an additional reduction; one must not treat a bound on printed ones as a bound on steps.